GUIA DO ALUNO

Servomotores: do primeiro movimento ao braço robótico

Controle, calibração e automação com Arduino e ESP32

Servo: Tower Pro SG90 (micro servo 9g) • **Placas:** Arduino Uno/Nano → ESP32

Projeto final: Servo Mini Arm (impressão 3D) operando em modo “linha de produção”

Carga horária sugerida: 6 encontros de 100 min

Espaço Cria — 2026

Sumário

1. Bem-vindo ao curso
2. Conheça o servo SG90
3. Segurança e cuidados
4. Atividade 1 — Primeiro movimento (varredura)
5. Atividade 2 — Controlando com o potenciômetro
6. Atividade 3 — Calibrando o servo
7. Atividade 4 — Migrando para o ESP32
8. Atividade 5 — Imprimindo e montando o Servo Mini Arm
9. Atividade 6 — Linha de produção: gravar e reproduzir
10. Para ir além
11. Glossário

Anexo 1 — Folha de calibração A: pulsos do servo
Anexo 2 — Folha de calibração B: limites mecânicos do braço
Anexo 3 — Checklist antes de ligar

1. Bem-vindo ao curso

Neste curso você vai aprender a controlar **servomotores** — o mesmo tipo de motor usado em robôs, drones, impressoras 3D e automação industrial. Começamos com um único servo no Arduino, aprendemos a **calibrar** seu movimento com um potenciômetro, migramos o mesmo projeto para o **ESP32** e terminamos montando um **braço robótico impresso em 3D** que grava e repete movimentos sozinho, como uma máquina numa linha de produção.

O caminho que vamos percorrer

Encontro	O que você faz	Placa	Sketch
1	Entende como o servo funciona e faz o primeiro movimento (varredura)	Arduino	01_sweep
2	Controla o ângulo com um potenciômetro e calibra os limites reais do servo	Arduino	02_pot_servo, 03_calibracao
3	Migra o projeto para o ESP32 e descobre o que muda	ESP32	04_pot_servo_esp32
4	Imprime e monta o Servo Mini Arm; centraliza os servos antes de montar	—	01_sweep (para centralizar)
5	Controla o braço com 4 potenciômetros (“marionete”) e define limites mecânicos	ESP32	05_braco_4pots_esp32
6	Grava poses e coloca o braço para trabalhar sozinho (teach & repeat)	ESP32	06_gravar_reproduzir_esp32

Como usar este guia

Cada atividade tem: **materiais → ligações → código → entenda o código → experimente → desafios**. Não pule o “experimente”: é mexendo no código que se aprende. Anote suas medidas nas **folhas de calibração** do final do guia.

Todos os sketches estão na pasta `aula-servos/codigo/`. Os códigos impressos aqui são exatamente os mesmos arquivos.

2. Conheça o servo SG90

O **Tower Pro SG90** é um micro servo de 9 gramas. Diferente de um motor comum, que só gira, o servo vai até um **ângulo** que você pede e fica lá, segurando a posição. Dentro dele há um motor DC, engrenagens de redução, um potenciômetro que mede a posição do eixo e um circuito que compara “onde estou” com “onde deveria estar”.

Característica	Valor do SG90	O que isso significa para você
Tensão de operação	4,8 a 6 V	Alimente com 5 V. Nunca com 3,3 V (fraco) nem 9 V (queima).
Torque	≈ 1,8 kgf·cm (4,8 V)	Levanta ~1,8 kg pendurado a 1 cm do eixo. Suficiente para o mini braço.
Velocidade	≈ 0,1 s / 60°	Percorre 0→180° em cerca de 0,3 s se você mandar direto.

Característica	Valor do SG90	O que isso significa para você
Curso	≈ 180° (0 a 180)	Não gira continuamente. Forçar além do limite quebra as engrenagens.
Sinal de controle	Pulso de 500 a 2400 µs a 50 Hz	É a largura do pulso que define o ângulo (ver abaixo).
Corrente	~10 mA parado, 100–250 mA movendo, até 650 mA travado	Um servo pode vir do USB; quatro precisam de fonte externa.
Fios	Marrom = GND, Vermelho = 5 V, Laranja = sinal	A ordem é sempre essa em servos Tower Pro.
Peso / engrenagens	9 g / plástico (nylon)	Leve e barato; trate com cuidado.

Como o servo entende o ângulo: o pulso de controle

A cada **20 ms** (50 vezes por segundo) o Arduino envia um pulso no fio laranja. O servo mede **quanto tempo o pulso fica em nível alto** e converte em ângulo:

Largura do pulso	Ângulo (aprox.)	Posição
500 µs (0,5 ms)	0°	Extremo esquerdo
1000 µs (1,0 ms)	45°	
1500 µs (1,5 ms)	90°	Centro (posição neutra)
2000 µs (2,0 ms)	135°	
2400 µs (2,4 ms)	180°	Extremo direito

A biblioteca `Servo` faz essa conversão por você: `servo.write(90)` gera pulsos de ~1500 µs. Quando quisermos controle fino, usaremos `servo.writeMicroseconds(1500)` diretamente. **Cada SG90 é um pouco diferente** — por isso existe a atividade de calibração.

3. Segurança e cuidados

- **Nunca force o eixo do servo com a mão** com ele energizado nem gire além dos batentes. As engrenagens são de plástico.
- **GND comum:** todo circuito com fonte externa precisa ter o GND da fonte ligado ao GND da placa. Sem isso o servo treme ou não responde.
- **Um servo** pode ser alimentado pelo 5 V do Arduino/USB. **Dois ou mais** (o braço tem quatro) exigem **fonte externa de 5 V, no mínimo 2 A**.
- No **ESP32, o potenciômetro vai no 3V3**, nunca no 5 V: os pinos de entrada só suportam 3,3 V.
- Desligue a alimentação antes de mexer nos fios. Confira as ligações duas vezes antes de ligar.
- Se um servo esquentar, ficar “roncando” ou vibrando forçando um batente, **desligue imediatamente** e revise os limites no código.
- Mantenha dedos e cabelos longe do braço quando ele estiver em modo automático.

4. Atividade 1 — Primeiro movimento (varredura)

Materiais

- 1 Arduino Uno ou Nano com cabo USB
- 1 servo SG90
- 3 jumpers macho-macho
- Computador com Arduino IDE 2

Ligações

Fio do servo	Cor	Arduino
GND	Marrom	GND
Alimentação	Vermelho	5V
Sinal	Laranja	Pino 9

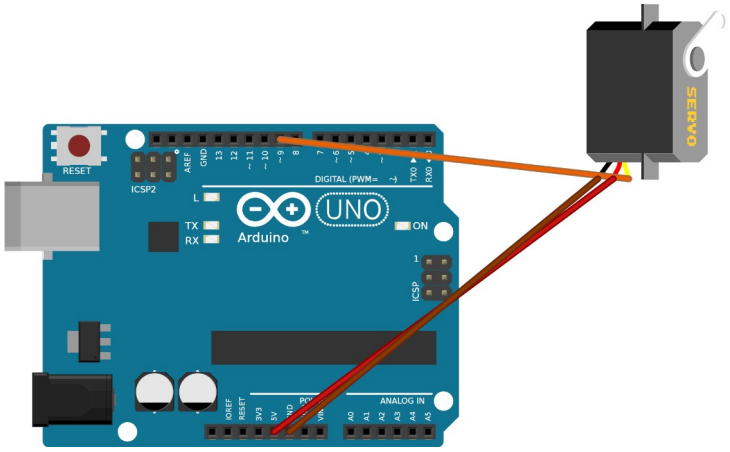


Figura 1 — Atividade 1 — servo SG90 ligado diretamente ao Arduino Uno (marrom = GND, vermelho = 5V, laranja = pino 9).

Código

01_sweep.ino

```
/*
  01 - Primeiro movimento: varredura (sweep)
  Placa: Arduino Uno/Nano
  Servo: Tower Pro SG90 (micro servo 9g)

  Ligacoes do SG90:
    fio MARROM  -> GND
    fio VERMELHO-> 5V
    fio LARANJA -> pino digital 9 (sinal)

  O que aprender: a biblioteca Servo gera o pulso de controle (50 Hz);
  write(angulo) recebe 0..180 graus.
*/
#include <Servo.h>

const int PINO_SERVO = 9;
Servo servo;

void setup() {
```

```
  // 500 e 2400 sao os pulsos (em microssegundos) para 0 e 180 graus do SG90.
  // O padrao da biblioteca (544..2400) tambem funciona, mas nao usa todo o curso.
  servo.attach(PINO_SERVO, 500, 2400);
}

void loop() {
  for (int ang = 0; ang <= 180; ang++) {   // vai de 0 a 180
    servo.write(ang);
    delay(15);                              // 15 ms por grau = ~2,7 s de curso
  }
  for (int ang = 180; ang >= 0; ang--) {   // volta de 180 a 0
    servo.write(ang);
    delay(15);
  }
}
```

Entenda o código

- `#include <Servo.h>` carrega a biblioteca que gera os pulsos. `Servo servo;` cria um objeto para o nosso motor.
- `servo.attach(9, 500, 2400)` diz em que pino está o sinal e quais pulsos correspondem a 0° e 180°.
- O `for` sobe de 0 a 180 mandando `servo.write(ang)` um grau por vez, com `delay(15)` entre eles — é isso que dá o movimento suave.
- Sem o `delay`, o servo receberia 180 comandos em menos de 1 ms e pularia direto para o fim.

Experimente

1. Mude `delay(15)` para `delay(2)` e depois para `delay(50)`. O que acontece com a velocidade?
2. Troque os limites do `for` para 45 e 135. O servo agora só varre o meio do curso.
3. Deixe só o primeiro `for` e coloque `delay(1000)` depois dele. Por que o servo volta “de uma vez”?

Desafios

1. **Desafio 1.1** — Faça o servo parar 1 segundo em 0°, 90° e 180° (três posições fixas, em loop).
2. **Desafio 1.2** — Faça a ida rápida e a volta lenta.
3. **Desafio 1.3** — Ligue um LED no pino 13 que acende enquanto o servo está indo e apaga enquanto volta.

5. Atividade 2 — Controlando com o potenciômetro

Agora **você** vai definir o ângulo girando um potenciômetro de 10 kΩ. É o mesmo princípio de um volante, de um controle de PlayStation ou do knob de volume.

Materiais extras

- 1 potenciômetro 10 kΩ (linear)
- 1 protoboard
- 5 jumpers

Ligações

Componente	Terminal	Arduino
Potenciômetro	Pino da esquerda	GND
Potenciômetro	Pino do meio (cursor)	A0
Potenciômetro	Pino da direita	5V
Servo	Marrom / Vermelho / Laranja	GND / 5V / Pino 9

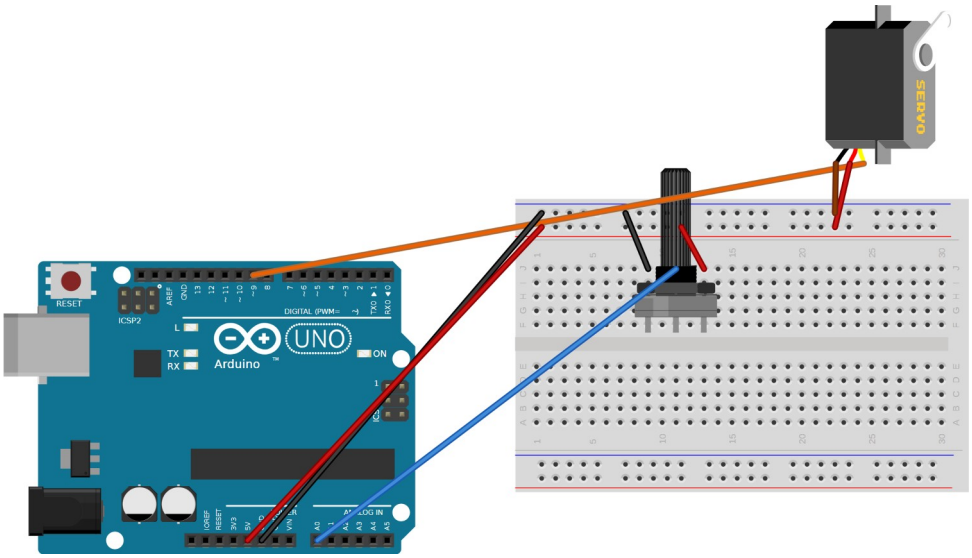


Figura 2 — Atividade 2 — potenciômetro na protoboard (GND e 5V pelas trilhas superiores, cursor em A0) e servo alimentado pelas mesmas trilhas.

O potenciômetro é um **divisor de tensão**: com o cursor no meio, A0 recebe 2,5 V; numa ponta, 0 V; na outra, 5 V. O Arduino converte isso em um número de **0 a 1023** (10 bits).

Código

02_pot_servo.ino

```
/*
  02 - Controlando o servo com um potenciometro de 10k
  Placa: Arduino Uno/Nano

  Ligacoes:
    Potenciometro 10k: pino da esquerda -> GND, pino do meio -> A0, pino da direita -> 5V
```

```
    SG90: marrom -> GND, vermelho -> 5V, laranja -> pino 9

  O que aprender: analogRead (0..1023) e a funcao map() para converter
  a leitura em angulo. O Monitor Serial mostra o que esta acontecendo.
*/
#include <Servo.h>

const int PINO_SERVO = 9;
const int PINO_POT   = A0;
Servo servo;

void setup() {
  Serial.begin(9600);
  servo.attach(PINO_SERVO, 500, 2400);
}

void loop() {
  int leitura = analogRead(PINO_POT);            // 0..1023
  int angulo  = map(leitura, 0, 1023, 0, 180);   // converte para 0..180
  servo.write(angulo);

  Serial.print("pot = ");
  Serial.print(leitura);
  Serial.print("  ->  angulo = ");
  Serial.println(angulo);
  delay(20);   // o servo precisa de tempo entre comandos (periodo de 20 ms)
}
```

Entenda o código

- `analogRead(A0)` devolve 0..1023.
- `map(leitura, 0, 1023, 0, 180)` faz uma regra de três: converte a faixa 0..1023 para 0..180.
- `Serial.begin(9600)` + `Serial.print` mostram os números no **Monitor Serial** (lupa no canto da IDE, 9600 baud). Use isso sempre que algo não funcionar: ver os valores é a melhor forma de depurar.

Experimente

1. Abra o Monitor Serial e gire o pot devagar. Anote em que leitura o servo começa a se mover e em qual para.
2. Inverta o sentido: `map(leitura, 0, 1023, 180, 0)`.
3. Limite o curso: `map(leitura, 0, 1023, 30, 150)`. Para que serviria isso num braço robótico?
4. Troque `delay(20)` por `delay(200)`. O servo fica “travado”? Por quê?

Desafios

1. **Desafio 2.1** — Crie uma “zona morta”: só mova o servo se o novo ângulo diferir do anterior em mais de 2°. (Dica: guarde o último ângulo numa variável.)
2. **Desafio 2.2** — Use dois potenciômetros e dois servos (pinos 9 e 10, A0 e A1).
3. **Desafio 2.3** — Faça o servo seguir o pot, mas com movimento suave: aproxime 1° por ciclo em vez de pular (essa técnica será usada no braço).

6. Atividade 3 — Calibrando o servo

A biblioteca assume que 500 µs = 0° e 2400 µs = 180°, mas na prática cada SG90 tem seus próprios limites. Se mandarmos um pulso além do batente, o servo **força** e pode se danificar; se ficarmos aquém, perdemos curso. Calibrar é **medir** os pulsos reais de cada extremo e usá-los no `attach()`.

Código

03_calibracao.ino

```
/*
  03 - Calibrando o servo: descobrindo os pulsos reais de 0 e 180 graus
  Placa: Arduino Uno/Nano

  Cada SG90 e um pouco diferente. Em vez de "write(angulo)", aqui usamos
  writeMicroseconds(pulso) e o potenciometro varre de 400 a 2600 us.
  Anote no Monitor Serial:
    - o menor pulso em que o servo PARA de se mover no inicio do curso (limite 0 graus)
    - o maior pulso em que ele PARA no fim do curso (limite 180 graus)
  Cuidado: se o servo "vibrar" ou "roncar" forcando o batente, volte o pot
  imediatamente. Forcar o fim de curso danifica as engrenagens.

  Ligacoes: iguais ao sketch 02.
*/
#include <Servo.h>

const int PINO_SERVO = 9;
const int PINO_POT   = A0;
const int PULSO_MIN_TESTE = 400;    // abaixo do minimo tipico (500)
const int PULSO_MAX_TESTE = 2600;   // acima do maximo tipico (2400)
Servo servo;

void setup() {
  Serial.begin(9600);
  servo.attach(PINO_SERVO, PULSO_MIN_TESTE, PULSO_MAX_TESTE);
  Serial.println("Gire o pot devagar. Anote o pulso onde o servo para em cada extremo.");
}

void loop() {
  int leitura = analogRead(PINO_POT);
  int pulso   = map(leitura, 0, 1023, PULSO_MIN_TESTE, PULSO_MAX_TESTE);
  servo.writeMicroseconds(pulso);

  Serial.print("pulso = ");
  Serial.print(pulso);
  Serial.println(" us");
  delay(50);
}

/*
  Depois de anotar, use os valores no attach() dos proximos sketches:
     servo.attach(9, PULSO_0_GRAUS, PULSO_180_GRAUS);
  Exemplo tipico de um SG90 real: attach(9, 560, 2350)
*/
```

Procedimento

1. Carregue o sketch e abra o Monitor Serial.
2. Gire o pot **devagar** desde uma ponta. Observe o servo: em algum momento ele **para de acompanhar** ou começa a vibrar — esse é o batente.
3. Volte um pouco até ele ficar quieto e anote o pulso: esse é o seu **pulso de 0°**.

4. Faça o mesmo na outra ponta: **pulso de 180°**.
5. Marque no eixo do servo (com caneta) a posição de 90° = média dos dois pulsos. Você vai precisar dela na montagem do braço.
6. Preencha a **Folha de calibração A** (Anexo 1) para cada servo que o grupo tiver. Identifique cada servo com uma etiqueta numerada.

Use os valores medidos daqui para frente

Em todos os próximos sketches, troque `attach(pino, 500, 2400)` pelos seus valores, por exemplo `attach(9, 560, 2350)`. Assim `write(0)` e `write(180)` passam a ser exatamente os extremos seguros do **seu** servo.

Desafios

1. **Desafio 3.1** — Meça o **tempo** que o servo leva de 0° a 180° usando `millis()` antes e depois do movimento. Compare com o valor da tabela (0,1 s/60°).
2. **Desafio 3.2** — Compare os pulsos de dois servos diferentes. Qual a diferença em microssegundos? Em graus?

7. Atividade 4 — Migrando para o ESP32

O **ESP32** é mais rápido, tem Wi-Fi e Bluetooth e muito mais memória — é o que usaremos no braço. Mas ele **não é um Arduino Uno**, e algumas coisas mudam:

Item	Arduino Uno	ESP32	Consequência
Lógica	5 V	3,3 V	Pot entre 3V3 e GND. Servo aceita sinal de 3,3 V, mas é alimentado com 5 V (pino VIN).
ADC (analogRead)	10 bits: 0..1023	12 bits: 0..4095	Mude o `map()`.
Pinos analógicos	A0..A5	GPIO 32..39 (ADC1)	GPIO 34, 35, 36, 39 são só entrada. ADC2 não funciona com Wi-Fi.
Biblioteca de servo	`Servo.h` (nativa)	`ESP32Servo.h` (instalar)	Gerenciador de Bibliotecas → ESP32Servo.
Serial	9600 típico	115200 típico	Ajuste o Monitor Serial.
Gravação	Automática	Às vezes é preciso segurar BOOT	Se der “Failed to connect”, segure BOOT ao iniciar o upload.

Preparando a IDE

1. Arquivo → Preferências → URLs adicionais: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`
2. Ferramentas → Placa → Gerenciador de Placas → instale **esp32 by Espressif Systems**.
3. Ferramentas → Placa → **ESP32 Dev Module**.
4. Sketch → Incluir biblioteca → Gerenciar bibliotecas → **ESP32Servo** (Kevin Harrington).
5. Se a porta COM não aparecer, o computador precisa do driver **CP210x** ou **CH340** (pergunte ao professor).

Ligações

Componente	Terminal	ESP32
Pot	esquerda / meio / direita	GND / GPIO 34 / **3V3**
Servo	marrom / vermelho / laranja	GND / **VIN (5 V)** / GPIO 13

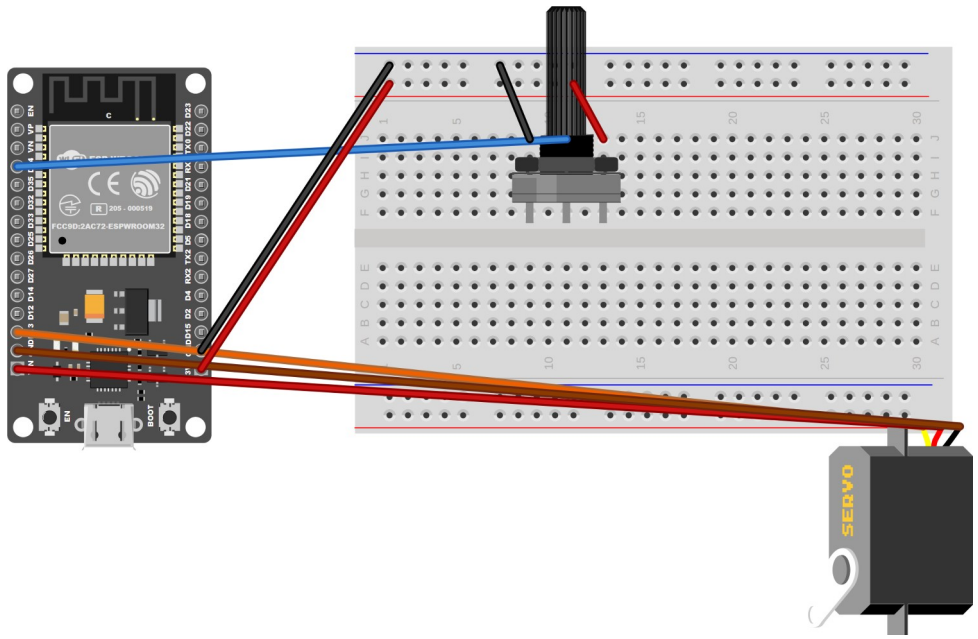


Figura 3 — Atividade 4 — no ESP32 o potenciômetro usa as trilhas em **3V3**; o servo é alimentado pelo **VIN** (5 V do USB) e recebe o sinal do GPIO 13.

Código

04_pot_servo_esp32.ino

```
/*
  04 - O mesmo controle por potenciometro, agora no ESP32
  Placa: ESP32 Dev Module (Arduino IDE: esp32 by Espressif)
  Biblioteca: ESP32Servo (Gerenciador de Bibliotecas -> "ESP32Servo")

  Diferencas em relacao ao Arduino Uno:
    1. A biblioteca Servo padrao NAO funciona no ESP32 -> usar ESP32Servo.
    2. O ADC do ESP32 tem 12 bits: analogRead vai de 0 a 4095 (nao 1023).
    3. O ESP32 trabalha em 3,3 V. O potenciometro deve ser ligado entre 3V3 e GND
       (NUNCA em 5 V, pois o pino ADC nao aguenta mais que 3,3 V).
    4. O sinal do servo em 3,3 V e aceito pelo SG90, mas a ALIMENTACAO do servo
       deve ser 5 V (pino VIN/5V do ESP32 quando ligado por USB, ou fonte externa).
    5. Use pinos do ADC1 para o pot (GPIO 32..39). ADC2 nao funciona com Wi-Fi ligado.

  Ligacoes:
    Pot 10k: esquerda -> GND, meio -> GPIO 34, direita -> 3V3
    SG90: marrom -> GND, vermelho -> VIN (5 V), laranja -> GPIO 13
*/
#include <ESP32Servo.h>

const int PINO_SERVO = 13;
const int PINO_POT   = 34;   // ADC1_CH6 (somente entrada)
Servo servo;

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);                 // 0..4095
  analogSetAttenuation(ADC_11db);           // permite ler ate ~3,3 V

  ESP32PWM::allocateTimer(0);               // reserva um timer para os servos
  servo.setPeriodHertz(50);                 // 50 Hz = periodo de 20 ms
  servo.attach(PINO_SERVO, 500, 2400);      // use os valores da SUA calibracao
}

void loop() {
  int leitura = analogRead(PINO_POT);            // 0..4095
  int angulo  = map(leitura, 0, 4095, 0, 180);
```

```
  servo.write(angulo);

  Serial.printf("pot = %4d  ->  angulo = %3d\n", leitura, angulo);
  delay(20);
}
```

Experimente e desafios

1. Abra o Monitor Serial a 115200. Confirme que a leitura vai até ~4095.
2. Você vai notar que o ADC do ESP32 **não é perfeitamente linear** nas pontas. Compare com o Arduino.
3. **Desafio 4.1** — Repita a calibração (sketch 03) no ESP32. Você precisa alterar quais linhas?
4. **Desafio 4.2** — Faça a média de 8 leituras do ADC antes do `map()`. O movimento fica mais estável?

8. Atividade 5 — Imprimindo e montando o Servo Mini Arm

O braço que vamos montar é o **Servo Mini Arm**, de LucaDilo, disponível gratuitamente no MakerWorld (modelo 1290483). Ele tem **4 graus de liberdade**: giro da base, ombro, cotovelo e uma garra que pega objetos de até 5 cm.

Lista de materiais do braço

Item	Qtd	Observação
Peças impressas em 3D	1 conjunto	Perfil de impressão “Mini Braço Servo” no MakerWorld (PLA, 0,2 mm)
Servo SG90 (180°)	4	O projeto original usa 1 servo de rotação contínua na base; com um SG90 comum a base gira ±90°, o que basta para a nossa linha de produção
Parafusos M3×12 (cabeça chata)	~8	Pivôs das juntas
Parafusos M2×5	~8	Fixação dos “horns” (braços do servo) nas peças
Parafusos do kit do servo	4 kits	Os parafusinhos que vêm com o SG90 prendem o horn no eixo
Cabos extensores de servo (20 cm)	2–4	Para os servos do cotovelo/garra chegarem à base
ESP32 + protoboard + jumpers	1	
Potenciômetros 10 kΩ	4	Um por junta
Fonte 5 V ≥ 2 A + capacitor 470–1000 µF	1	Alimentação dos servos
2 botões (push button)	2	Gravar / Play (Atividade 6)

ANTES de montar: centralize os servos!

Um servo montado fora do centro vai bater na base ou não alcançar a posição. Para cada servo, com o sketch **01_sweep** modificado para `servo.write(90)` fixo (ou o pulso central que você mediu na calibração), energize, espere ele ir ao centro, **desligue e só então** encaixe o horn na posição indicada no guia de montagem. Não gire o eixo com a mão depois disso.

Montagem

1. Baixe e siga o **Servo Mini Arm Assembly Sheet.pdf** (aba “Arquivos & Guias” do MakerWorld).
2. Remova rebarbas e teste o encaixe das juntas a seco antes de parafusar.
3. Passe os cabos dos servos pelos canais das peças **antes** de fechar as juntas (as versões atualizadas de 21/05/2025 têm passagem para os cabos).
4. Monte a base com o compartimento de eletrônica (“Base com Compartimento”), que guarda o ESP32.
5. Aperte os parafusos M3 só até a junta girar sem folga — apertado demais trava o servo.
6. Com o braço montado, ligue **somente** o servo da base com o sketch 02 e verifique o curso. Repita junta por junta.

Ligações completas (4 servos + 4 pots)

Junta	Servo (sinal)	Potenciômetro (cursor)	Limites iniciais (min–max)
Base	GPIO 13	GPIO 34	0–180
Ombro	GPIO 12	GPIO 35	20–160
Cotovelo	GPIO 14	GPIO 32	10–170
Garra	GPIO 27	GPIO 33	40–120

Todos os pots entre **3V3 e GND**. Fios vermelhos dos servos no **+5 V da fonte externa**, marrons no GND da fonte, e **o GND da fonte ligado ao GND do ESP32**. Capacitor entre +5 V e GND, perto dos servos.

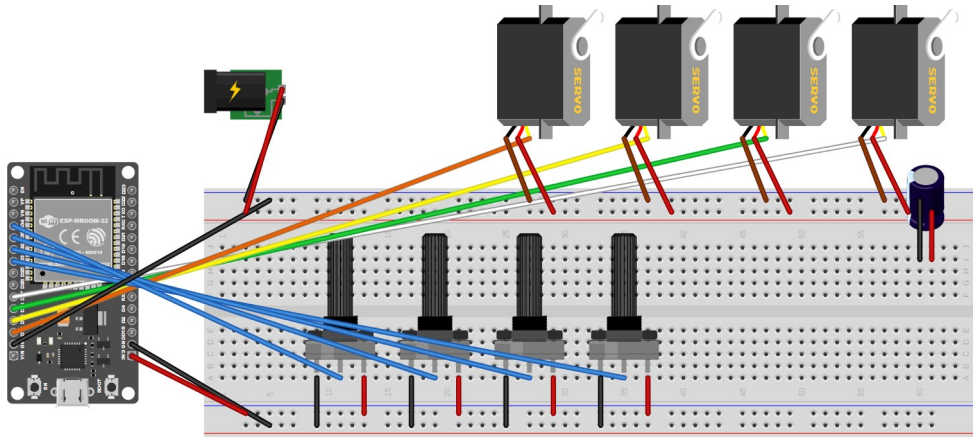


Figura 4 — Atividade 5 — braço completo: trilhas **superiores** = 5 V da fonte externa (servos + capacitor 470 µF); trilhas **inferiores** = 3V3 do ESP32 (potenciômetros). O fio preto do GND do ESP32 até a trilha superior é o **GND comum**.

Código

05_braco_4pots_esp32.ino

```
/*
  05 - Servo Mini Arm: 4 servos controlados por 4 potenciometros (modo "marionete")
  Placa: ESP32 Dev Module + biblioteca ESP32Servo
  Braco: Servo Mini Arm (LucaDilo, MakerWorld 1290483)
         base (giro), ombro, cotovelo, garra

  ALIMENTACAO: 4 servos SG90 podem puxar mais de 1 A ao mesmo tempo.
  NAO alimente pelo 3V3 nem pelo USB do ESP32. Use fonte externa de 5 V (>= 2 A)
  ligada aos fios vermelhos dos servos, com o GND da fonte unido ao GND do ESP32.
  Um capacitor eletrolitico de 470 a 1000 uF entre 5 V e GND ajuda a evitar resets.

  Ligacoes (sinal dos servos / pots):
    base    -> GPIO 13   pot base    -> GPIO 34
    ombro   -> GPIO 12   pot ombro   -> GPIO 35
    cotovelo-> GPIO 14   pot cotovelo-> GPIO 32
    garra   -> GPIO 27   pot garra   -> GPIO 33
    Todos os pots entre 3V3 e GND.
*/
#include <ESP32Servo.h>

const int N = 4;
const char* NOME[N]      = {"base", "ombro", "cotovelo", "garra"};
const int PINO_SERVO[N]  = {13, 12, 14, 27};
const int PINO_POT[N]    = {34, 35, 32, 33};

// Limites mecanicos de cada junta (ajuste apos montar o braco!).
// Evitam que o braco bata na base ou que a garra force alem do fechamento.
const int ANG_MIN[N] = {  0,  20,  10,  40 };
```

```
const int ANG_MAX[N] = {180, 160, 170, 120 };

Servo servos[N];
int anguloAtual[N];

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);
  analogSetAttenuation(ADC_11db);
  ESP32PWM::allocateTimer(0);
  ESP32PWM::allocateTimer(1);

  for (int i = 0; i < N; i++) {
    servos[i].setPeriodHertz(50);
    servos[i].attach(PINO_SERVO[i], 500, 2400);
    anguloAtual[i] = (ANG_MIN[i] + ANG_MAX[i]) / 2;   // comeca no meio do curso
    servos[i].write(anguloAtual[i]);
  }
  delay(500);
}

void loop() {
  for (int i = 0; i < N; i++) {
    int leitura = analogRead(PINO_POT[i]);
    int alvo    = map(leitura, 0, 4095, ANG_MIN[i], ANG_MAX[i]);

    // Movimento suave: aproxima 1 grau por ciclo em vez de pular direto.
    // Isso reduz o pico de corrente e o "tranco" do braco.
    if (alvo > anguloAtual[i]) anguloAtual[i]++;
    else if (alvo < anguloAtual[i]) anguloAtual[i]--;
    servos[i].write(anguloAtual[i]);
  }

  static unsigned long ultimoPrint = 0;
  if (millis() - ultimoPrint > 250) {
    ultimoPrint = millis();
    for (int i = 0; i < N; i++) Serial.printf("%s=%3d  ", NOME[i], anguloAtual[i]);
    Serial.println();
  }
  delay(10);
}
```

Entenda o código

- Os pinos, nomes e limites ficam em **vetores** (`PINO_SERVO[N]`, `ANG_MIN[N]`…). Assim um único `for` trata as 4 juntas.
- `ANG_MIN`/`ANG_MAX` são os **limites mecânicos**: o `map()` já converte o pot para essa faixa, então o pot no extremo nunca força a junta.
- O movimento suave (`anguloAtual[i]++`) evita trancos e picos de corrente que reiniciam o ESP32.
- `ESP32PWM::allocateTimer(0)` e `(1)`: o ESP32 usa timers de hardware para os pulsos; 4 servos precisam de 2 timers.

Definindo os limites do SEU braço

1. Comece com limites largos e mova cada junta devagar com o pot.
2. Quando a peça encostar em algo (ou a garra fechar), anote o ângulo mostrado no Monitor Serial e recue 5°.
3. Preencha a **Folha de calibração B** (Anexo 2) e atualize `ANG_MIN`/`ANG_MAX` no código.

4. A garra merece atenção: o `ANG_MAX` deve ser o ângulo em que ela **encosta** no objeto, não o em que força. Servo forçando = corrente alta = engrenagem quebrada.

Desafios

1. **Desafio 5.1** — Adicione um botão que leva o braço à "pose de descanso" (todas as juntas no centro).
2. **Desafio 5.2** — Mude a velocidade de aproximação (1° por ciclo) para 3° e para 0,5° (dica: use `float`). Quando cada uma é melhor?
3. **Desafio 5.3** — Faça a garra fechar automaticamente quando um LDR (fotoresistor) detectar um objeto entre os dedos.

9. Atividade 6 — Linha de produção: gravar e reproduzir

Robôs industriais são programados assim: um operador **guia o braço** pelas posições do trabalho (“teach”), o robô **grava** cada pose e depois **repete** o ciclo milhares de vezes (“repeat”). Você vai implementar exatamente isso.

O que muda no hardware

- 2 botões: **GRAVAR** no GPIO 25 e **PLAY/PARAR** no GPIO 26 (um lado no GPIO, outro no GND — usamos `INPUT_PULLUP`).
- Uma “esteira” improvisada: marque no papel a posição de **pegar** e a de **soltar** um objeto pequeno (tampinha, cubo de 2 cm, borracha).

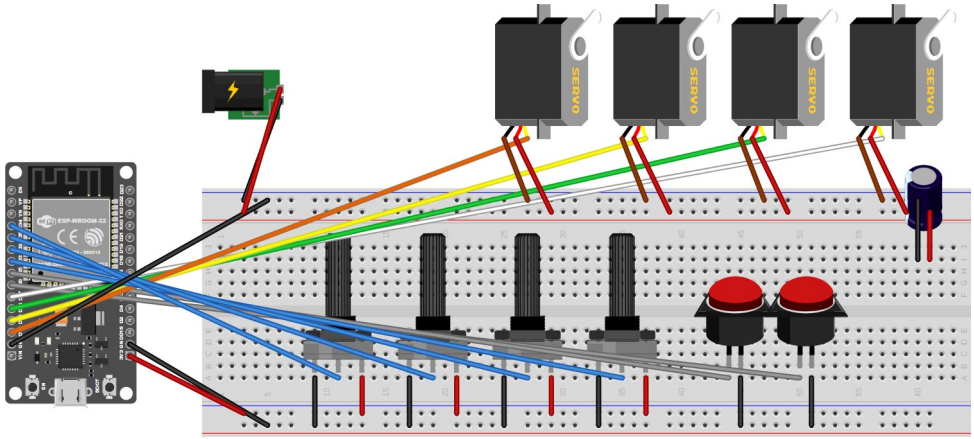


Figura 5 — Atividade 6 — mesma montagem da Atividade 5 com os botões GRAVAR (GPIO 25) e PLAY (GPIO 26); cada botão liga o GPIO ao GND (usamos INPUT_PULLUP).

Código

06_gravar_reproduzir_esp32.ino

```
/*
  06 - Linha de producao: gravar poses e reproduzir automaticamente
  Placa: ESP32 Dev Module + ESP32Servo + Preferences (memoria flash)

  Ideia (igual a um robo industrial em modo "teach"):
    1. MODO MANUAL: voce posiciona o braco com os 4 potenciometros.
    2. Aperta o botao GRAVAR: a pose atual (4 angulos) entra na lista.
    3. Repete para cada etapa do ciclo (pegar, levantar, girar, soltar, voltar...).
    4. Aperta PLAY: o braco repete a sequencia sozinho, em loop, como numa esteira.
    5. As poses ficam salvas na flash: sobrevivem a desligar o ESP32.

  Botoes (com INPUT_PULLUP: um lado no GPIO, outro no GND):
    GRAVAR -> GPIO 25     PLAY/PARAR -> GPIO 26     APAGAR TUDO -> segurar GRAVAR 3 s

  Comandos pelo Monitor Serial (115200) tambem funcionam:
    g = gravar pose   p = play/parar   l = listar   x = apagar tudo

  Servos e pots: mesmas ligacoes do sketch 05.
*/
#include <ESP32Servo.h>
#include <Preferences.h>

const int N = 4;
const int PINO_SERVO[N] = {13, 12, 14, 27};
const int PINO_POT[N]   = {34, 35, 32, 33};
const int ANG_MIN[N]    = {  0,  20,  10,  40 };
const int ANG_MAX[N]    = {180, 160, 170, 120 };
const int PINO_BTN_GRAVAR = 25;
```

```
const int PINO_BTN_PLAY   = 26;

const int MAX_POSES = 20;
struct Pose { uint8_t ang[N]; };
Pose poses[MAX_POSES];
int numPoses = 0;

// Parametros do ciclo automatico (mexa aqui para mudar o "ritmo da linha")
const int PASSO_MS       = 15;    // ms por grau ao interpolar: menor = mais rapido
const int PAUSA_POSE_MS  = 600;   // tempo parado em cada pose (ex.: fechar a garra)

Servo servos[N];
int anguloAtual[N];
bool modoPlay = false;
Preferences prefs;

// ---------- persistencia ----------
void salvar() {
  prefs.begin("braco", false);
  prefs.putInt("n", numPoses);
  prefs.putBytes("poses", poses, sizeof(Pose) * numPoses);
  prefs.end();
}
void carregar() {
  prefs.begin("braco", true);
  numPoses = prefs.getInt("n", 0);
  if (numPoses > MAX_POSES) numPoses = 0;
  if (numPoses > 0) prefs.getBytes("poses", poses, sizeof(Pose) * numPoses);
  prefs.end();
}

// ---------- movimento ----------
void irPara(const uint8_t alvo[N]) {          // interpola todas as juntas juntas
  bool chegou = false;
  while (!chegou) {
    chegou = true;
    for (int i = 0; i < N; i++) {
      if (anguloAtual[i] < alvo[i])      { anguloAtual[i]++; chegou = false; }
      else if (anguloAtual[i] > alvo[i]) { anguloAtual[i]--; chegou = false; }
      servos[i].write(anguloAtual[i]);
    }
    delay(PASSO_MS);
  }
}

void lerPots() {
  for (int i = 0; i < N; i++) {
    int alvo = map(analogRead(PINO_POT[i]), 0, 4095, ANG_MIN[i], ANG_MAX[i]);
    if (alvo > anguloAtual[i]) anguloAtual[i]++;
    else if (alvo < anguloAtual[i]) anguloAtual[i]--;
    servos[i].write(anguloAtual[i]);
  }
}

// ---------- acoes ----------
void gravarPose() {
  if (numPoses >= MAX_POSES) { Serial.println("Lista cheia!"); return; }
  for (int i = 0; i < N; i++) poses[numPoses].ang[i] = anguloAtual[i];
  numPoses++;
  salvar();
  Serial.printf("Pose %d gravada: ", numPoses);
  for (int i = 0; i < N; i++) Serial.printf("%d ", anguloAtual[i]);
  Serial.println();
}
void listar() {
  Serial.printf("%d pose(s):\n", numPoses);
  for (int p = 0; p < numPoses; p++) {
    Serial.printf("  #%2d  base=%3d ombro=%3d cotovelo=%3d garra=%3d\n", p + 1,
                  poses[p].ang[0], poses[p].ang[1], poses[p].ang[2], poses[p].ang[3]);
  }
}
void apagarTudo() { numPoses = 0; salvar(); Serial.println("Poses apagadas."); }
```

```
// ---------- botoes com debounce simples ----------
bool apertou(int pino) {
  static unsigned long ultimo[40] = {0};
  if (digitalRead(pino) == LOW && millis() - ultimo[pino] > 300) {
    ultimo[pino] = millis();
    return true;
  }
  return false;
}

void setup() {
  Serial.begin(115200);
  pinMode(PINO_BTN_GRAVAR, INPUT_PULLUP);
  pinMode(PINO_BTN_PLAY,   INPUT_PULLUP);
  analogReadResolution(12);
  analogSetAttenuation(ADC_11db);
  ESP32PWM::allocateTimer(0);
  ESP32PWM::allocateTimer(1);
  for (int i = 0; i < N; i++) {
    servos[i].setPeriodHertz(50);
    servos[i].attach(PINO_SERVO[i], 500, 2400);
    anguloAtual[i] = (ANG_MIN[i] + ANG_MAX[i]) / 2;
    servos[i].write(anguloAtual[i]);
  }
  carregar();
  Serial.println("g=gravar  p=play/parar  l=listar  x=apagar");
  listar();
}

void loop() {
  // --- comandos serial ---
  if (Serial.available()) {
    char c = Serial.read();
    if (c == 'g') gravarPose();
    if (c == 'p') modoPlay = !modoPlay;
    if (c == 'l') listar();
    if (c == 'x') apagarTudo();
  }
  // --- botoes ---
  if (apertou(PINO_BTN_PLAY)) { modoPlay = !modoPlay; Serial.println(modoPlay ? "PLAY" : "PARADO"); }
  if (digitalRead(PINO_BTN_GRAVAR) == LOW) {
    unsigned long t0 = millis();
    while (digitalRead(PINO_BTN_GRAVAR) == LOW) {
      if (millis() - t0 > 3000) { apagarTudo(); while (digitalRead(PINO_BTN_GRAVAR) == LOW); return; }
    }
    gravarPose();
  }

  if (modoPlay && numPoses > 0) {
    for (int p = 0; p < numPoses && modoPlay; p++) {
      irPara(poses[p].ang);
      delay(PAUSA_POSE_MS);
      if (apertou(PINO_BTN_PLAY) || (Serial.available() && Serial.read() == 'p')) {
        modoPlay = false; Serial.println("PARADO");
      }
    }
  } else {
    lerPots();          // modo manual: segue os potenciometros
    delay(10);
  }
}
```

Entenda o código

- `struct Pose { uint8_t ang[4]; }` guarda uma posição completa do braço. `poses[MAX_POSES]` é a lista de etapas.
- **Modo manual** (`lerPots()`): igual ao sketch 05. **Modo play**: percorre as poses com `irPara()`, que interpola todas as juntas ao mesmo tempo.

- `Preferences` grava a lista na **memória flash**: você pode desligar o ESP32 e as poses continuam lá (como o EEPROM do Arduino).
- `apertou()` faz **debounce**: ignora repetições do botão por 300 ms.
- Os parâmetros `PASSO_MS` e `PAUSA_POSE_MS` definem o **ritmo da linha**.

Roteiro: seu primeiro ciclo de produção

1. Apague poses antigas (`x` no Serial ou segure GRAVAR 3 s).
2. Pose 1 — **descanso**: braço recolhido, garra aberta. GRAVAR.
3. Pose 2 — **aproximação**: sobre o objeto, garra aberta. GRAVAR.
4. Pose 3 — **pegar**: desce e fecha a garra. GRAVAR.
5. Pose 4 — **levantar**: sobe com o objeto. GRAVAR.
6. Pose 5 — **transportar**: gira a base até o destino. GRAVAR.
7. Pose 6 — **soltar**: desce e abre a garra. GRAVAR.
8. Pose 7 — volta ao descanso. GRAVAR.
9. Aperte PLAY. Reponha o objeto no ponto inicial a cada ciclo e observe a repetibilidade.

Por que gravar a pose de "aproximação" antes de "pegar"?
Se o braço for direto do descanso para "pegar", a interpolação em linha reta no espaço das juntas pode passar **por dentro** do objeto ou da mesa. Poses intermediárias ("waypoints") definem o caminho seguro. É assim que se programa qualquer robô.

Desafios

1. **Desafio 6.1** — Reduza `PASSO_MS` até o braço começar a derrubar o objeto. Qual o limite? O que isso diz sobre velocidade × precisão?
2. **Desafio 6.2** — Faça a pausa ser maior só na pose em que a garra fecha (dica: guarde um tempo por pose no `struct`).
3. **Desafio 6.3** — Conte os ciclos completados e mostre no Serial: "Peças produzidas: N".
4. **Desafio 6.4** — Adicione um sensor (LDR ou botão de fim de curso) que só inicia o ciclo quando há peça na posição de pegar — como um sensor de presença numa esteira real.
5. **Desafio 6.5** — Envie cada ciclo concluído por MQTT para o painel do laboratório (tópico `sensores/braco`) e veja o contador no Grafana.
6. **Desafio 6.6 (Uno)** — Faça o mesmo no Arduino Uno com `EEPROM` (veja `07_gravar_reproduzir_uno.ino`). Que limitações aparecem?

10. Para ir além

- **Controle por Wi-Fi:** um servidor web no ESP32 com sliders para cada junta (biblioteca `WebServer` + WebSocket).
- **Controle por Python:** ler o Serial com `pyserial` e enviar poses a partir de um script ou de um painel Streamlit.
- **Node-RED:** um dashboard com botões Gravar/Play e um gráfico dos ângulos ao vivo.
- **Visão:** uma câmera + OpenCV para detectar a cor da peça e escolher o destino.
- **Cinemática:** calcular os ângulos a partir de uma coordenada (x, y) desejada da garra.

11. Glossário

Termo	Significado
PWM / pulso	Sinal digital que fica um tempo em nível alto e outro em baixo. No servo, a largura do pulso define o ângulo.
Servo	Motor com controle de posição em malha fechada: vai a um ângulo e fica lá.
ADC	Conversor analógico-digital: transforma tensão em número (10 bits no Uno, 12 no ESP32).
map()	Função que converte um valor de uma faixa para outra (regra de três).
Calibração	Medir os valores reais do seu hardware e ajustar o software a eles.
Debounce	Ignorar os múltiplos “cliques” elétricos que um botão gera ao ser apertado uma vez.
Interpolação	Gerar posições intermediárias entre duas poses para o movimento ser contínuo.
Waypoint	Pose intermediária gravada só para definir um caminho seguro.
Teach & repeat	Método de programar robôs guiando-os manualmente pelas posições e gravando-as.
Brownout	Queda de tensão (ex.: servo puxando corrente) que reinicia o microcontrolador.
GND comum	Referência de 0 V compartilhada entre fonte, placa e sensores; obrigatória para os sinais fazerem sentido.

Anexo 1 — Folha de calibração A: pulsos do servo

Sketch: `03_calibracao`. Um servo por linha. Centro = (pulso 0° + pulso 180°) ÷ 2.

Servo nº	Pulso 0° (µs)	Pulso 180° (µs)	Centro (µs)	Usado em (junta)	Obs.
1					
2					
3					
4					
5					
6					

Linha para o `attach()`: `servo.attach(pino, _____, _____);`

Anexo 2 — Folha de calibração B: limites mecânicos do braço

Sketch: `05_braco_4pots_esp32`. Mova devagar, anote onde encosta, recue 5°.

Junta	Servo nº	Ângulo onde encosta (mín.)	ANG_MIN adotado	Ângulo onde encosta (máx.)	ANG_MAX adotado
Base					
Ombro					
Cotovelo					
Garra					

Linhas para o código: `const int ANG_MIN[N] = { ___, ___, ___, ___ };` `const int ANG_MAX[N] = { ___, ___, ___, ___ };`

Anexo 3 — Checklist antes de ligar

- [ ] Fio marrom de todos os servos no GND; vermelho no +5 V; laranja nos GPIOs certos
- [ ] GND da fonte externa ligado ao GND da placa
- [ ] Potenciômetros entre 3V3 e GND (ESP32) ou 5V e GND (Uno)
- [ ] Nenhum servo com o horn montado fora do centro
- [ ] Limites `ANG_MIN`/`ANG_MAX` conferidos
- [ ] Monitor Serial aberto na velocidade certa (9600 Uno / 115200 ESP32)
- [ ] Objeto de teste leve e área do braço livre